Analysis Draft

Grace Yanzito, Lucy Dobson, Hannah Bregel

1 Correctness and Asymptotic Analysis

We analyze the depth-first search with the Dijkstra check and the branch-and-bound step, as implemented in `find_ranked_trails`.

1.1 Setup and Notation

The trail network is a directed graph $G = (V, E)$ with $n = |V|$ points and $m = |E|$ directed segments, each with a length $w(u, v) > 0$ (a 0 in the matrix means no segment). The origin is s, the budget is D, each point may be visited at most M times ($M = 2$ in our runs), the list holds the best k routes ($k = 25$), Δ is the largest number of segments leaving one point, and $w_{\min}$ is the shortest segment. A *route* is a walk $W = (v_0 = s, v_1, \ldots, v_\ell)$. It is *feasible* if:

(R1) it never immediately turns around ($v_{i+1} = v_{i-1}$) unless v_i has only one outgoing segment;

(R2) no directed segment is used twice;

(R3) no point appears more than M times;

(R4) its total length is at most D.

A *loop* is a feasible route with $v_\ell = s$ and $v_i \neq s$ for $0 < i < \ell$ (the search stops the first time it returns to s). Routes are ranked by number of new points, then fresh miles, then fewer repeat miles, with ties broken by the order in which the search finds them. The search is:

```
DFS(curr, dist, newCount):
  if loop mode and curr = s and path has a step: record(path); return
  if point-to-point mode and path has a step:    record(path)
  if list is full and newCount + reachable(curr, D - dist) < worst.newCount:
      return                                     # branch and bound
  for each segment (curr -> nxt, w), unvisited points first:
      skip if it breaks R1, R2 or R3, or if dist + w > D
      skip if loop mode and dist + w + home[nxt] > D     # Dijkstra check
      extend path by nxt; DFS(nxt, dist + w, newCount + [nxt is new]); undo
```

Here `home[v]` is the shortest distance from v back to s, and `reachable` counts unvisited, not-yet-hiked points v with $\mathrm{sp}(curr, v) + home[v] \leq D - dist$ (just $\mathrm{sp}(curr, v) \leq D - dist$ in point-to-point mode), where sp is the shortest distance between two points.

1.2 Correctness

Lemma 1 (Dijkstra)**.** *For every point v, `home[v]` is the length of the shortest walk from v to s, so every walk from v to s has length at least `home[v]`. The same holds for* $\mathrm{sp}(u, v)$.

Proof. This is the standard correctness of Dijkstra's algorithm for positive segment lengths. The code runs it on the reversed segments, which gives distances *to* the target. □

Lemma 2 (Termination)**.** *The search makes finitely many calls.*

Proof. By (R2) a path on the DFS stack has at most m steps, and each call tries at most Δ extensions, so the recursion tree is finite. □

Lemma 3 (Soundness)**.** *Every route the search records is feasible, and in loop mode is a loop.*

Proof. The path starts as (s). An extension is made only after its checks for (R1)–(R3) and the budget pass, so every path on the stack is feasible. In loop mode a route is recorded only when the path returns to s, and the search returns immediately, so s appears only at the two ends. □

Lemma 4 (The Dijkstra check loses nothing)**.** *Every feasible loop W is reached and recorded, up to mirror image, by the search without branch and bound.*

Proof. Let $W = (v_0, \ldots, v_\ell)$ be a feasible loop and W_i its prefix ending at v_i. Each prefix satisfies (R1)–(R3) and is no longer than W, so those checks and the budget check pass at every step. For the Dijkstra check, the rest of W is a walk from v_i to s, so by Lemma 1 its length is at least $home[v_i]$, and therefore $\text{length}(W_i) + home[v_i] \le \text{length}(W) \le D$. By induction on i, the search reaches each prefix, because at every call it tries every extension that passes the checks, and it records W at the end. If the mirror image of W is found first, only that one is kept. □

Lemma 5 (Branch and bound loses nothing)**.** *Suppose the list is full with worst new-point count c^*, and the test fires at a partial route P ending at curr, so* $\text{new}(P) + \text{reachable} < c^*$*. Then no completion of P can enter the list.*

Proof. Let Q be any completion of P that satisfies the rules. A point counted as new in Q but not in P is unvisited and not hiked before, and getting from *curr* to it costs at least $\text{sp}(curr, v)$ (and, in loop mode, getting home afterward costs at least $home[v]$). Both must fit in the remaining budget, so v is counted in reachable. Hence $\text{new}(Q) \le \text{new}(P) + \text{reachable} < c^*$. Routes are ranked by new points first, so Q ranks strictly below the current worst entry. That entry only improves as the search continues, so Q never enters the list. □

Lemma 6 (Top-k list)**.** *After each call to record, the list holds the* $\min(k, r)$ *highest-ranked of the r routes recorded so far.*

Proof. By induction on r. While fewer than k routes are held, every new route is added. Once the list has k routes, its worst entry is at the root of the heap, and a new route replaces it only if it ranks strictly higher, which keeps the k best. □

Theorem 1. *If no call or time cap is triggered, the list returned by the search contains the k highest-ranked feasible loops (feasible routes, in point-to-point mode), where a loop and its mirror image count as one.*

Proof. By Lemmas 2 and 3 the search terminates and records only feasible routes. By Lemma 4 every feasible loop is recorded unless branch and bound removes it, and by Lemma 5 anything removed could not have entered the list. By Lemma 6 the list holds the best k of what is recorded. In point-to-point mode the same argument holds with the Dijkstra check omitted and every prefix recorded. □

Remark. (i) The Dijkstra check is necessary but not sufficient: the shortest way home may itself break (R1)–(R3), so some explored branches never finish a loop. (ii) If a cap triggers, the theorem no longer applies and the output is only a good list, not a guaranteed best one. (iii) The theorem proves the search correct for the route model (R1)–(R4) and the scoring rule; whether that model matches how a hiker thinks about a loop is a modeling question that testing against real trail maps has to answer.

1.3 Running Time

Table 1 summarizes the cost of each part. The matrix costs $O(n^2)$ to build, and each Dijkstra run

Part	Time
Build the matrix and neighbor lists	$O(n^2)$
One Dijkstra run (`dijkstra_to`)	$O(n^2 + m \log n)$
Shortest distances between all pairs (for branch and bound)	$O(n^3 + nm \log n)$
One DFS call (sort options, bound check, record, M constant)	$O(n \log n)$
Number of DFS calls T	$\leq \Delta^{L+1}$, where $L = \min\{m,\ Mn - 1,\ \lfloor D/w_{\min} \rfloor\}$

Table 1: Running time of each part of the search ($\Delta \geq 2$).

scans one matrix row per point plus a heap operation per segment. A single DFS call sorts at most Δ options ($O(\Delta \log \Delta) \subseteq O(n \log n)$), runs the bound check in $O(n)$, and when it records a route copies and scores a path of length at most $Mn - 1$, which is $O(n)$ for constant M.

Every DFS call corresponds to a distinct feasible partial route, and a route of length ℓ has at most Δ^ℓ possibilities. A route can have at most m steps by (R2), at most $Mn - 1$ steps by (R3), and at most $D/w_{\min}$ steps by (R4). Summing over lengths up to L gives $T \leq \Delta^{L+1}$, so the total time is

$$O\big(n^3 + \Delta^{L+1}\, n \log n\big)\,.$$

This bound is exponential in n, and it is loose for our data: on the Silver Falls graph the search made 275 calls for loops and 625 for point-to-point routes, nowhere near the bound.

Proposition 1. *Let G_k be a chain of k "diamonds": points $j_0 = s, j_1, \ldots, j_k$, and for each i two points a_i, b_i each joined to both j_{i-1} and j_i (so $n = 3k + 1$), with every segment of length 1, $M = 2$, and $D = 4k$. The search makes at least $2^k = 2^{(n-1)/3}$ DFS calls. We prove this for the search with branch and bound turned off, and Table 2 gives the counts for both versions.*

Proof. Consider the 2^k outbound walks $s \to x_1 \to j_1 \to x_2 \to \cdots \to x_k \to j_k$ with $x_i \in \{a_i, b_i\}$. Each is feasible: no point is visited twice, and there are no turnarounds. At j_i the Dijkstra check reads $2i + 2i \leq 4k$, and at x_i it reads $(2i - 1) + (2i - 1) \leq 4k$, so both pass. These are 2^k distinct paths, hence 2^k distinct DFS calls. □

Remark. Table 2 shows the call counts on G_k. Both versions exceed 2^k, and the version with branch and bound still grows by roughly a factor of 2 per diamond, so branch and bound does not remove the exponential growth on this family. We have not proved this for the version with branch and bound.

k	n	2^k	Calls (no B&B)	Calls (with B&B)
2	7	4	41	41
3	10	8	169	169
4	13	16	689	527
5	16	32	2,785	1,485
6	19	64	11,201	2,487
7	22	128	44,929	4,777
8	25	256	179,969	9,313

Table 2: DFS calls for loop search on the diamond chain G_k ($D = 4k$, all segments of length 1). B&B means branch and bound.

What the exponential worst case means. An exponential worst case is expected for an exact method. The orienteering problem (maximize the reward collected under a distance limit) and the longest simple path problem are both known to be NP-hard (Golden, Levy, and Vohra, 1987, for orienteering). Our problem allows repeated visits, so we do not claim a hardness proof for it, but it suggests that no polynomial-time exact algorithm should be expected. The caps give a worst-case time guarantee at the cost of exactness: stopping after C calls costs $O(n^3 + C\,n \log n)$ in total.